

Design and Analysis of Algorithms

Induction-Based Algorithms

Dr. Golzari

University of Hormozgan

Fall 1405

Induction as a Design Tool

From Induction Proofs to Algorithms

Induction *proves* statements

To show $P(n)$ for all $n \geq n_0$:

1. **Base:** verify $P(n_0)$ directly.
2. **Step:** from $P(k)$ derive $P(k+1)$.

Induction *designs* algorithms

To solve an instance of size n :

1. **Base:** solve the smallest instance directly.
2. **Step:** reduce size $k+1$ to size k , extend.

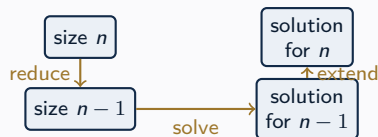
Design mantra

Assume the problem of size $n - 1$ is already solved — find a cheap way to extend that solution to size n .

The inductively designed algorithm carries its own correctness proof: **the proof is the algorithm.**

The Induction Template

Inductive proof	Algorithmic counterpart
base case $P(n_0)$	solve smallest instance directly
hypothesis $P(k)$	"the solver works for size k "
step $P(k) \Rightarrow P(k+1)$	shrink to size k , extend
conclusion	correct for every size



If the reduction may jump to *any* smaller size, use **strong induction**: assume $P(j)$ for all $j \leq k$.

Warm-Up: Sum of Array Elements

Algorithm

```
1: Sum( $A, n$ ):  
2: if  $n = 0$  then  
3:   return 0  
4: return Sum( $A, n - 1$ ) +  $A[n - 1]$ 
```

Correctness (induction)

- **Base:** $n = 0$: the empty sum is 0.
- **Step:** the recursive call returns $A[0] + \dots + A[n - 2]$; adding $A[n - 1]$ completes it.
- The hypothesis *is* the recursive call.

Running time

$T(n) = T(n - 1) + \Theta(1) \Rightarrow T(n) = \Theta(n)$: the induction scheme itself tells us how the work grows.

How Do We Shrink an Instance?

Shrinking rule	Classical examples	Typical time
by one	insertion sort, topological sort	usually $\Theta(n^2)$
by a constant factor	binary search, fake coin	$\Theta(\log n)$
variable amount	Euclid's gcd, selection	data dependent

- The shrinking rule fixes both the **induction scheme** and the **recurrence** for the running time.

How Do We Shrink an Instance?

Shrinking rule	Classical examples	Typical time
by one	insertion sort, topological sort	usually $\Theta(n^2)$
by a constant factor	binary search, fake coin	$\Theta(\log n)$
variable amount	Euclid's gcd, selection	data dependent

- The shrinking rule fixes both the **induction scheme** and the **recurrence** for the running time.
- Today's case study (min & max) shrinks the instance **by two**: one intra-pair comparison replaces two scans.

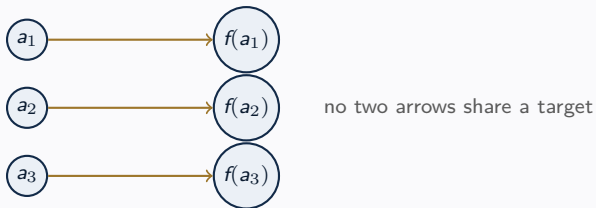
Classification follows the “decrease-and-conquer” taxonomy (Levitin).

One-to-One Mappings

One-to-One Mappings: Definition

Injective (one-to-one) mapping

$f: A \rightarrow B$ is **one-to-one** if different elements of A are mapped to different elements of B :
 $a_1 \neq a_2 \implies f(a_1) \neq f(a_2)$.



- If every element of B is hit too, f is a **bijection**: a perfect matching between A and B .
- A bijection forces $|A| = |B|$ — the key to many counting arguments.

One-to-One Mappings as a Proof Tool

- **Counting.** To count a complicated set A , build a bijection with a set B that is easy to count.

One-to-One Mappings as a Proof Tool

- **Counting.** To count a complicated set A , build a bijection with a set B that is easy to count.
- **Pigeonhole principle.** No one-to-one mapping exists from a bigger set to a smaller one: 13 people and 12 months $\Rightarrow$ two share a birth month.

One-to-One Mappings as a Proof Tool

- **Counting.** To count a complicated set A , build a bijection with a set B that is easy to count.
- **Pigeonhole principle.** No one-to-one mapping exists from a bigger set to a smaller one: 13 people and 12 months $\Rightarrow$ two share a birth month.
- **Lower bounds.** If every comparison settles at most one distinct “unit of information” and m units are needed, then at least m comparisons are required.

One-to-One Mappings as a Proof Tool

- **Counting.** To count a complicated set A , build a bijection with a set B that is easy to count.
- **Pigeonhole principle.** No one-to-one mapping exists from a bigger set to a smaller one: 13 people and 12 months $\Rightarrow$ two share a birth month.
- **Lower bounds.** If every comparison settles at most one distinct “unit of information” and m units are needed, then at least m comparisons are required.
- **Assigning roles.** If every input element is mapped to exactly one role, no element is processed twice and none is forgotten.

Coming up

The pairwise min–max algorithm is built — and proved optimal — entirely out of one-to-one mapping arguments.

Case Study: Minimum and Maximum

Finding Minimum and Maximum

Problem

Given an array $A[0..n-1]$, find both its minimum and maximum elements.

Example:

$$A = [8, 3, 12, 5, 1, 9], \quad \min(A) = 1, \quad \max(A) = 12$$

First question

How many comparisons do we really need to find both answers?

The Obvious Approach

Scan the array once and compare every new element with both the current minimum and the current maximum.

```
1:  $min \leftarrow A[0], \quad max \leftarrow A[0]$ 
2: for  $i \leftarrow 1$  to  $n - 1$  do
3:   if  $A[i] < min$  then
4:      $min \leftarrow A[i]$ 
5:   if  $A[i] > max$  then
6:      $max \leftarrow A[i]$ 
```

Number of comparisons

Each of the remaining $n - 1$ elements may require two comparisons: $2(n - 1) = 2n - 2$.

Key Idea: Compare Elements in Pairs

Instead of comparing every element with both *min* and *max*, first compare two elements with each other.



- The smaller element of each pair can only improve *min*.
- The larger element of each pair can only improve *max*.

One-to-One Mapping of Candidates

For every pair (a, b) , exactly one element becomes a minimum candidate and exactly one a maximum candidate.

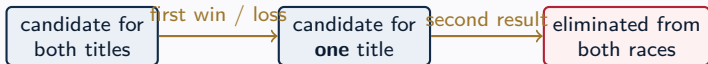


Why this helps

After the pair comparison each element has only one possible role, so we never compare it with both values.

Candidacies and Eliminations

Every element starts as a candidate for *both* titles; each comparison can strip at most one candidacy.



- **Winning** removes an element from the minimum race; **losing** removes it from the maximum race.
- Once out of both races an element is **useless**: never compare with it again.
- These first and second events are exactly what the lower-bound proof will charge to comparisons.

Pairwise Minimum–Maximum Algorithm

```
1:  $min \leftarrow max \leftarrow A[0], \quad start \leftarrow 1$   
2: if  $n$  is even then  
3:   compare  $A[0]$  with  $A[1]$ ;  $start \leftarrow 2$   
4:    $min \leftarrow \min(A[0], A[1]), \quad max \leftarrow \max(A[0], A[1])$   
5: for  $i \leftarrow start$  to  $n - 2$  in steps of 2 do  
6:   compare  $A[i]$  with  $A[i + 1]$   
7:    $small \leftarrow \min(A[i], A[i + 1]), \quad large \leftarrow \max(A[i], A[i + 1])$   
8:    $min \leftarrow \min(min, small), \quad max \leftarrow \max(max, large)$   
9: return ( $min, max$ )
```

Lines 7–9 are the **three** comparisons of each pair; the last line updates both answers at once. For even n the first pair seeds min and max ; for odd n the loop starts at index 1.

Claim

After processing k pairs, min and max are respectively the minimum and maximum among all processed elements.

- **Base case:** after the first pair, the smaller value is assigned to min and the larger value to max .

Claim

After processing k pairs, min and max are respectively the minimum and maximum among all processed elements.

- **Base case:** after the first pair, the smaller value is assigned to min and the larger value to max .
- **Inductive hypothesis:** assume the claim holds after processing k pairs.

Correctness by Induction

Claim

After processing k pairs, min and max are respectively the minimum and maximum among all processed elements.

- **Base case:** after the first pair, the smaller value is assigned to min and the larger value to max .
- **Inductive hypothesis:** assume the claim holds after processing k pairs.
- **Inductive step:** the next pair contributes one smaller and one larger value; comparing them with min and max preserves the claim.

Conclusion

By induction, the algorithm returns the correct minimum and maximum.

Comparison Count

Each processed pair costs exactly **three** comparisons: inside the pair, the smaller one against *min*, the larger one against *max*.

Method	Comparisons	Time
Direct scan	$2n - 2$	$O(n)$
Max first, then min of the rest	$2n - 3$	$O(n)$
Pairwise method, even n	$\frac{3n}{2} - 2$	$O(n)$
Pairwise method, odd n	$\frac{3(n-1)}{2}$	$O(n)$

Both pairwise rows equal $\lceil 3n/2 \rceil - 2$: about 25% fewer comparisons than the direct scan — and, as the next slide shows, this is *optimal*.

Lower Bound: $\lceil 3n/2 \rceil - 2$ Comparisons

Two facts per element

Every non-maximum element must **lose** once and every non-minimum element must **win** once: $2(n - 1)$ facts in total. Otherwise an unproven element could still be the max or the min.

Adversary counting (one-to-one charging)

A comparison settles *two* facts only when two *fresh* elements meet — possible at most $\lfloor n/2 \rfloor$ times; every other comparison settles at most one fact.

$$2n - 2 \leq 2\lfloor n/2 \rfloor + (c - \lfloor n/2 \rfloor) \implies c \geq \lceil 3n/2 \rceil - 2.$$

Conclusion

The pairwise algorithm matches this bound exactly: **it is optimal.**

Teaser: Minimum and Second Minimum

New problem

Find the minimum *and* the second minimum of $A[0..n-1]$. How few comparisons suffice?

- Anyone who lost to someone *other than* the minimum cannot be second smallest — the answer hides among the elements compared *directly with the minimum*.

Teaser: Minimum and Second Minimum

New problem

Find the minimum *and* the second minimum of $A[0..n-1]$. How few comparisons suffice?

- Anyone who lost to someone *other than* the minimum cannot be second smallest — the answer hides among the elements compared *directly with the minimum*.
- Run a **balanced knockout tournament**: the minimum then has at most $\lceil \log_2 n \rceil$ opponents.

$$\underbrace{(n-1)}_{\text{tournament}} + \underbrace{(\lceil \log_2 n \rceil - 1)}_{\text{min's opponents}} = n + \lceil \log_2 n \rceil - 2$$

Think about

Why is this below $\lceil 3n/2 \rceil - 2$? Because each loss now *records information* — a one-to-one log of the minimum's victims.

Summary: The Min/Max Case Study

- Pair the elements *before* touching *min* and *max*; each pair yields exactly one candidate for each title.

Summary: The Min/Max Case Study

- Pair the elements *before* touching *min* and *max*; each pair yields exactly one candidate for each title.
- Correctness follows by a clean induction over the pairs.

Summary: The Min/Max Case Study

- Pair the elements *before* touching *min* and *max*; each pair yields exactly one candidate for each title.
- Correctness follows by a clean induction over the pairs.
- Comparisons: from $2n - 2$ down to $\lceil 3n/2 \rceil - 2$ — and no algorithm can do better.

Summary: The Min/Max Case Study

- Pair the elements *before* touching *min* and *max*; each pair yields exactly one candidate for each title.
- Correctness follows by a clean induction over the pairs.
- Comparisons: from $2n - 2$ down to $\lceil 3n/2 \rceil - 2$ — and no algorithm can do better.
- Same $O(n)$ class, strictly less work.

Key idea

A good algorithm does not always change the asymptotic class; sometimes it cuts the exact amount of work inside the same class — and one-to-one counting arguments prove both correctness and optimality.

More Induction-Based Algorithms

Insertion Sort: Think Inductively

Inductive idea

- **Hypothesis:** the prefix $A[0..k-1]$ is already sorted.
- **Step:** insert $A[k]$ so that $A[0..k]$ becomes sorted.



Result: $[3, 5, 7, 12, 9]$ — the sorted prefix grew by one; the insertion *is* the extension step of the induction.

Insertion Sort: Algorithm and Analysis

```
1: for  $i \leftarrow 1$  to  $n - 1$  do  
2:    $key \leftarrow A[i]; \quad j \leftarrow i - 1$   
3:   while  $j \geq 0$  and  $A[j] > key$  do  
4:      $A[j + 1] \leftarrow A[j]$   
5:      $j \leftarrow j - 1$   
6:    $A[j + 1] \leftarrow key$ 
```

Complexity

- Worst case (reverse input):
 $\sum_{i=1}^{n-1} i = \frac{n(n-1)}{2} = \Theta(n^2)$.
- Best case (sorted input): $n - 1$ comparisons, $\Theta(n)$.

Properties

In-place, stable, excellent for nearly sorted data — correctness follows immediately from the inductive formulation.

Insertion Sort: Correctness by Induction

Claim $P(i)$

After iteration i of the **for** loop, the prefix $A[0..i]$ contains the original elements $A[0..i]$ in sorted order.

- **Base case:** before the loop ($i = 0$), a single element is trivially sorted.

Insertion Sort: Correctness by Induction

Claim $P(i)$

After iteration i of the **for** loop, the prefix $A[0..i]$ contains the original elements $A[0..i]$ in sorted order.

- **Base case:** before the loop ($i = 0$), a single element is trivially sorted.
- **Inductive step:** assume $A[0..i - 1]$ is sorted. The **while** loop shifts every element greater than *key* one position right and stops at the first smaller-or-equal element; placing *key* there keeps the prefix sorted.

Insertion Sort: Correctness by Induction

Claim $P(i)$

After iteration i of the **for** loop, the prefix $A[0..i]$ contains the original elements $A[0..i]$ in sorted order.

- **Base case:** before the loop ($i = 0$), a single element is trivially sorted.
- **Inductive step:** assume $A[0..i - 1]$ is sorted. The **while** loop shifts every element greater than *key* one position right and stops at the first smaller-or-equal element; placing *key* there keeps the prefix sorted.
- **Inner termination:** j strictly decreases and is bounded below by -1 .

Conclusion

By induction, after the final iteration $i = n - 1$ the whole array is sorted.

Preview: Topological Sorting by Induction

Problem

Given a DAG $G = (V, E)$, list its vertices so that every edge (u, v) has u before v .

- **Inductive idea:** a *source* (a vertex with no incoming edges) can safely be placed **first**: nothing needs to precede it.

Preview: Topological Sorting by Induction

Problem

Given a DAG $G = (V, E)$, list its vertices so that every edge (u, v) has u before v .

- **Inductive idea:** a *source* (a vertex with no incoming edges) can safely be placed **first**: nothing needs to precede it.
- Output the source, delete it, recurse on the smaller DAG — the instance shrank *by one vertex*.

Preview: Topological Sorting by Induction

Problem

Given a DAG $G = (V, E)$, list its vertices so that every edge (u, v) has u before v .

- **Inductive idea:** a *source* (a vertex with no incoming edges) can safely be placed **first**: nothing needs to precede it.
- Output the source, delete it, recurse on the smaller DAG — the instance shrank *by one vertex*.
- **Inductive proof:** putting a source first violates no edge; by hypothesis the algorithm sorts the remaining $|V| - 1$ vertices.

Question to think about

What happens if at some stage the remaining graph has no source? (The graph must contain a cycle — so this is also a cycle-detection algorithm.)

Wrap-Up

Key Takeaways

- **Induction is a design paradigm**, not only a proof technique: solve size $n - 1$, extend to size n .

Key Takeaways

- **Induction is a design paradigm**, not only a proof technique: solve size $n - 1$, extend to size n .
- **One-to-one mappings** turn intuition into precise counting: assigning roles, charging work, proving lower bounds.

Key Takeaways

- **Induction is a design paradigm**, not only a proof technique: solve size $n - 1$, extend to size n .
- **One-to-one mappings** turn intuition into precise counting: assigning roles, charging work, proving lower bounds.
- **Min/max**: pairwise comparisons achieve $\lceil 3n/2 \rceil - 2$ comparisons — and this is optimal.

Key Takeaways

- **Induction is a design paradigm**, not only a proof technique: solve size $n - 1$, extend to size n .
- **One-to-one mappings** turn intuition into precise counting: assigning roles, charging work, proving lower bounds.
- **Min/max**: pairwise comparisons achieve $\lceil 3n/2 \rceil - 2$ comparisons — and this is optimal.
- **Constants matter inside one O -class**: $2n - 2 \rightarrow 1.5n - 2$ is a 25% saving on a basic operation.

Next session

Divide and conquer: what changes when we split a problem into *several* subproblems instead of one?

1. Using a one-to-one mapping between “lost comparisons” and non-maximum elements, prove that finding only the maximum requires at least $n - 1$ comparisons.
2. Show that the two-phase method (maximum first, then the minimum of the remaining $n - 1$ elements) uses exactly $2n - 3$ comparisons.
3. Rewrite the pairwise min–max algorithm recursively on $A[0..n - 1]$ and prove it correct by strong induction.
4. Prove the correctness of insertion sort by strong induction and derive its best- and worst-case counts.
5. **(Challenge)** Prove that $n + \lceil \log_2 n \rceil - 2$ comparisons are also *necessary* for finding both the minimum and the second minimum.